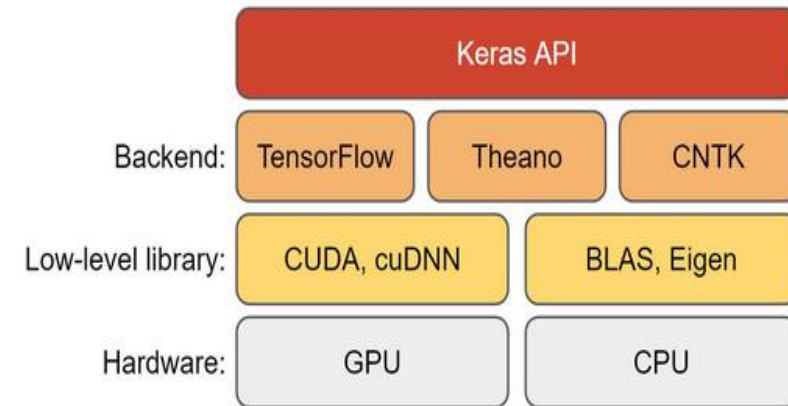


Linear Algebra-Scalars, Vectors, Matrices and Tensors-
Operations –Special kind of Matrices and Vectors-
Probability Basics-Gradient based optimization- Linear
Classifiers, Linear Machines with Hinge Loss-
Introduction to Neural Network-Activation functions-
Tensorflow , keras operations

Keras

- Keras is a high-level, deep learning API developed by Google for implementing neural networks.
- It is written in Python and is used to make the implementation of neural networks.
- It also supports multiple backend neural network computation.
- This makes Keras slower than other deep learning frameworks, but extremely beginner-friendly.
- Keras has features such as :
 - It runs smoothly on both CPU and GPU.
 - It supports almost all neural network models.
 - It is expressive, flexible, and apt for innovative research.
- It cannot handle low-level computations, so it uses the Backend library to resolve it.
- The different back ends supported by Keras are:
 - Tensorflow
 - Theano

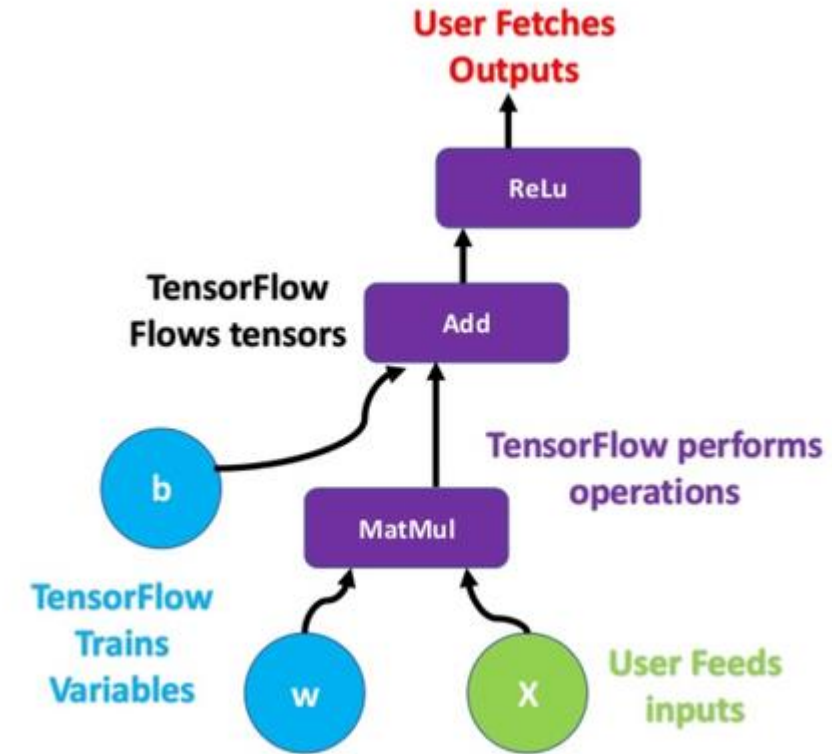


TensorFlow (Tensor framework)

- Tensor is a multidimensional array
- Flow is used to define the flow of data in operation.
- TensorFlow is an open-source library developed by Google primarily for deep learning and machine learning applications.

Tensors

- TensorFlow accepts data as multi-dimensional arrays of higher dimensions called tensors.
- Multi-dimensional arrays are useful to handle large amounts of data.
- TensorFlow works based on data flow graph architecture that contains nodes and edges.
- As the execution mechanism is in the form of graphs, it is much easier to execute TensorFlow code in a distributed manner across a cluster of computers while using GPUs.



TensorFlow (Tensor framework)

- The architecture of TensorFlow is divided into three parts:
- Preparing the data
- Creating the model
- Prepare the model by training it and estimating it.

Algorithms for TensorFlow

- TensorFlow 1.10 includes a built-in API for:
 - **LinearRegressor:** tf. Estimator
 - **Classification:** tf.estimator for LinearClassifier
 - **DL classification:** tf. estimator.DNNClassifier
 - **Wipe and deep learning:** tf. estimator
 - **DNNLinearCombinedClassifier :**tf. Estimator (booster tree regression).

TensorFlow (Tensor framework)

CPU Support

- `conda create --name tensorflow`
- `conda activate tensorflow`
- `conda install jupyter`
- `conda install scipy`
- `pip install tensorflow`
- `print("TensorFlow version:", tf.__version__)`

GPU support

- `conda create --name tensorflow-gpu`
- `conda activate tensorflow-gpu`
- `conda install jupyter`
- `conda install scipy`
- `pip install tensorflow-gpu`

TensorFlow (Tensor framework)

- `x_train, y_train), (x_test, y_test) = mnist.load_data()`
- `x_train, x_test = x_train / 255.0, x_test / 255.0`

Building a ML Model

- `model = tf.keras.models.Sequential([
 tf.keras.layers.Flatten(input_shape=(28, 28)),
 tf.keras.layers.Dense(128, activation='relu'),
 tf.keras.layers.Dropout(0.2),
 tf.keras.layers.Dense(10)
])`

Training and Evaluating your Model

- `model.fit(x_train, y_train, epochs=5)`
- `model.evaluate(x_test, y_test, verbose=2)`
- `probability_model = tf.keras.Sequential([ model, tf.keras.layers.Softmax()])`

TensorFlow (Tensor framework)

Build model

- `model = models.Sequential()`
- `model.add(layers.Dense(1, input_dim=1, activation='linear'))`

Get initial, untrained weights

- `print(model.layers[0].get_weights())`

Configure training process

- `model.compile(optimizer=optimizers.SGD(), loss='mse', metrics=['mse'])`

Train model

- `model.fit(x, y, batch_size=1, epochs=50, shuffle=False)`

Try predicting 1 value

- `model.predict([1.1])`

Plot fitted line versus actual data

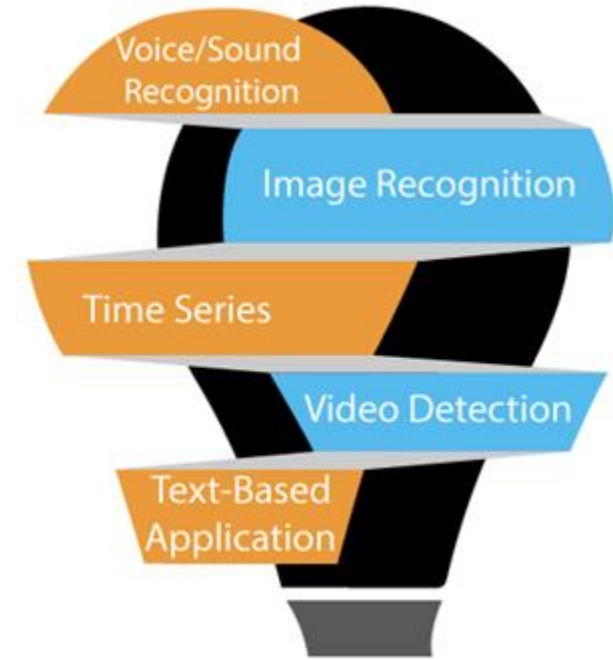
- `y_pred = model.predict(x)`

Get trained weights

- `print(model.layers[0].get_weights())`

Applications of TensorFlow (Tensor framework)

- Voice/Sound Recognition
- Image Recognition
- Time Series
- Video Detection
- Text-Based Applications



Tensorflow and Theano

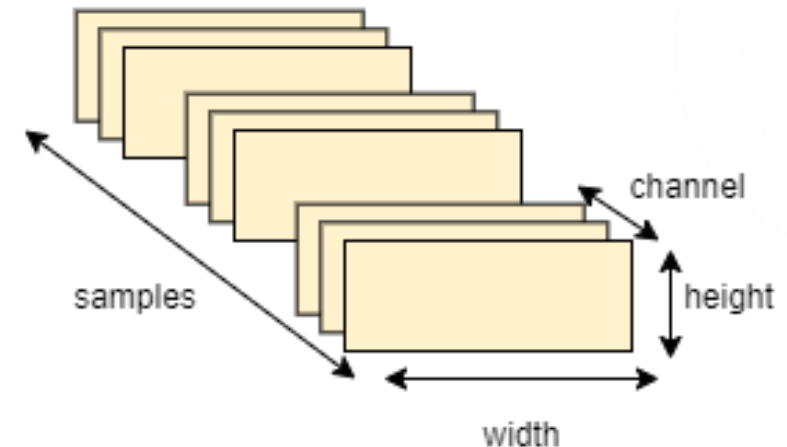
Tensorflow takes channels as the last format.

Theano uses channels-first format

The colour-depth axis is located at the end of the list of dimensions in the Google TensorFlow: (samples, height, width, colour-depth).

The batch axis is placed first, followed by the samples, colour-depth, height, and width axes by **Theano**.

The previous instances would be transformed into (32, 1, 64, 64) and (32, 3, 64, 64). Both file types are supported by the Keras framework



References

- <https://www.javatpoint.com/tensorflow-introduction>
- <https://www.analyticsvidhya.com/blog/2022/03/a-basic-introduction-to-tensorflow-in-deep-learning/>